Rapport de Sprint 1

Projet L3 : CV_LLM_app
Moteur de recherche sémantique de CV par IA

19 Décembre 2025

Étudiants :

Jaouad Kabouri-Ouzennou, Samy Sadeg, Nassim Benenes

Encadrés par :

Poncelet Pascal

1 Introduction

Ce rapport présente le Sprint 1 du projet **CV_LLM_app**, un moteur de recherche sémantique conçu pour comprendre le sens réel d'un CV grâce au Deep Learning. Notre approche repose sur trois piliers techniques :

- **Embeddings :** Vectorisation du texte pour capturer le sens des compétences au-delà des mots exacts.
- **Contexte :** Analyse contextuelle plutôt qu'une détection isolée de termes.
- **Architecture Transformer :** Utilisation de mécanismes d'attention pour pondérer les informations clés du candidat.

Notre objectif avec ce projet n'est pas seulement de créer un filtre, mais un véritable outil capable de mesurer la proximité réelle entre le parcours d'un candidat et les besoins d'un poste.

2 Compte rendu de la réunion avec l'encadrant

- **Date et lieu :** Mardi 16 décembre, 9h15-11h40, Bâtiment 5.212.
- **Participants :** Pascal Poncelet, Jaouad Kabouri, Samy Sadeg, Nassim Benenes, ainsi que les membres des autres projets TER de M. Poncelet.
- **Décisions prises :**
  - Développement d'une application web basée sur l'architecture Flask.
  - Utilisation initiale d'un modèle de langage (LLM) basé sur BERT, léger et fonctionnant en local.
  - Planification d'une phase d'expérimentation future pour maximiser la pertinence (via fine-tuning ou comparaison avec des approches classiques type *bag-of-words*).

3 Cahier des charges

À l'aide de notre encadrant, nous avons évalué, lors de la réunion au campus, la faisabilité du projet en fonction du temps imparti et de nos compétences. Notre objectif est de concevoir une application web capable d'automatiser le tri de candidatures. Le système doit "comprendre" le sens d'un CV pour mesurer sa proximité sémantique avec une offre d'emploi spécifique.

3.1 Périmètre retenu (MVP)

Pour le premier jalon fonctionnel, le périmètre se concentre sur :

- L'upload sécurisé de plusieurs CV en simultané.
- L'extraction brute du texte depuis des documents PDF.
- La vectorisation de ces textes ainsi que l'intégration d'un champ textuel simple pour décrire le profil recherché.

Une fois la demande de l'utilisateur et les CV vectorisés, nous calculons un score de pertinence. Ce score correspond au cosinus de l'angle θ formé par les deux vecteurs (Similarité Cosinus) :

$$Score = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

3.2 Contraintes techniques et choix technologiques

- **Langage :** Python (pour sa richesse en librairies).
- **Backend :** Flask (framework léger et robuste).
- **Extraction PDF :** Librairie **pdfplumber**.
- **LLM :** `all-MiniLM-L6-v2` (version optimisée de BERT).
- **Frontend :** HTML5/CSS et JavaScript (Fetch API).

4 Première architecture du projet

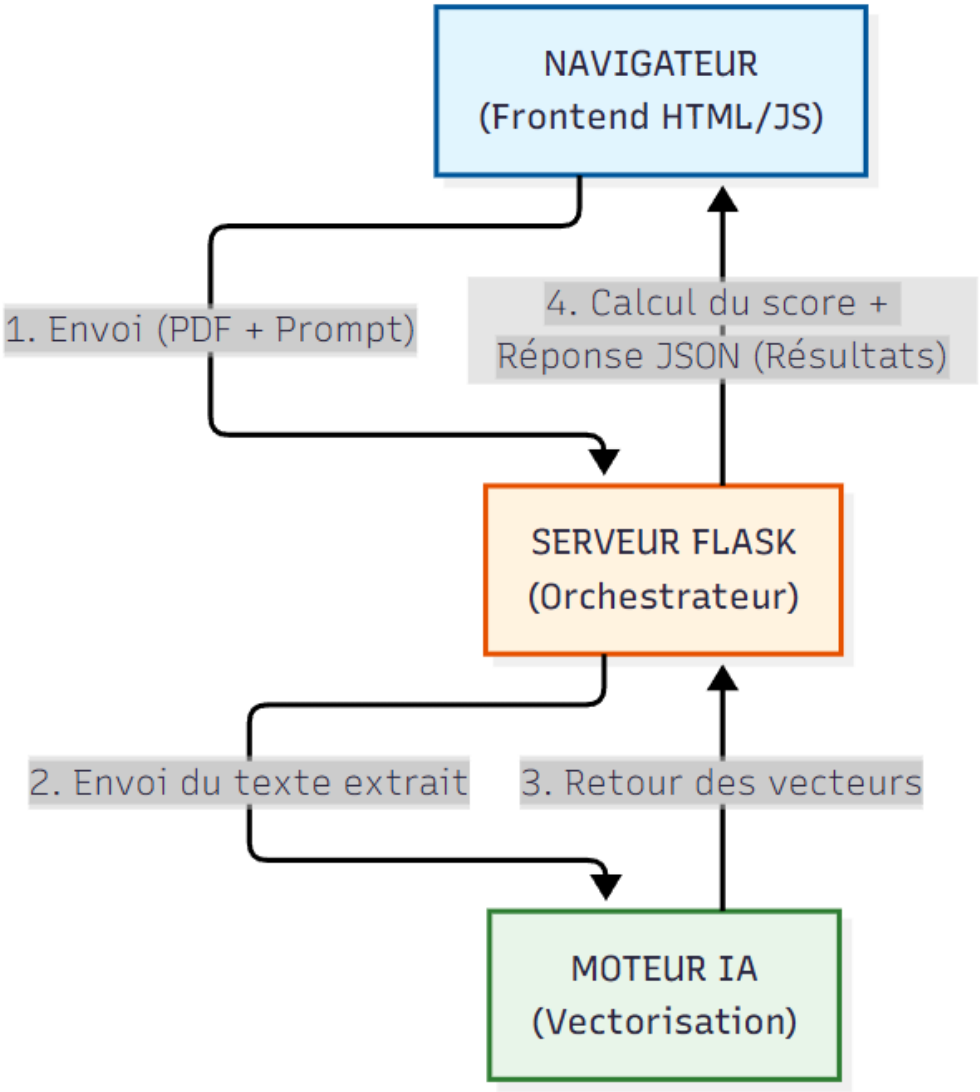

FIGURE 1 – Architecture globale et flux de données de l'application CV_LLM

Comme le détaille la **Figure 1**, le traitement s'effectue en quatre étapes :

1. **Envoi (Flux descendant) :** Tout d'abord, le **Navigateur** (en bleu) envoie les fichiers PDF et le prompt utilisateur vers le serveur via une requête POST.
2. **Extraction et Transmission :** Le **Serveur Flask** (en orange), qui agit comme orchestrateur, extrait le texte brut des PDF et le transmet au module d'intelligence artificielle.
3. **Vectorisation :** Le **Moteur IA** (en vert) transforme ce texte en représentations mathématiques (embeddings) et renvoie ces vecteurs au serveur.
4. **Calcul et Réponse (Flux montant) :** Enfin, le serveur utilise ces vecteurs pour effectuer le calcul de similarité (le score) et renvoie le tout au client sous format JSON pour l'affichage.

Cette structure permet de découpler la logique d'interface (Frontend) de la logique de calcul (Backend), facilitant ainsi la maintenance et les évolutions futures.

5 Organisation interne du groupe

5.1 Répartition des tâches pour le Sprint 1

— **Samy Sadeg :** En charge de la mise en place de Flask, développement du module de pré-traitement (nettoyage PDF) et intégration du moteur IA (Sentence-Transformers).

— **Jaouad Kabouri :** En charge de la gestion des flux de fichiers (système d'upload et stockage) et de sa liaison avec le serveur Flask.

— **Nassim Benenes :** En charge de l'Interface Utilisateur (Frontend), intégration du champ de saisie (prompt) et la mise en place d'un leaderboard pour les CV avec affichage du score de pertinence.

5.2 Outils et méthodologie

Nous avons utilisé **Python** (Flask) pour le backend, ainsi que **HTML5, CSS3 et JavaScript natif** pour le frontend. La communication est assurée via **Discord**. Le développement a été réalisé sur **VS Code** et **IntelliJ**, avec une centralisation du code sur le **Gitlab étudiant** (gestion par branches). L'IA générative a été utilisée ponctuellement comme outil d'aide à la correction du code CSS uniquement, ainsi que la recherche d'information.

6 Planning prévisionnel

FIGURE 2 – Diagramme de Gannt prévisionnel pour la planification des tâches

Comme l'illustre la **Figure 2**, le développement du projet s'articule autour de quatre grandes phases mensuelles, allant de la mise en place du MVP à la livraison finale.

- **12 Décembre - 19 Décembre (Sprint 1) :** Installation de l'environnement, développement de l'interface d'upload, gestion du stockage, nettoyage des données et première estimation de pertinence. (*Terminé*)
- **18 Janvier - 20 Février :** Amélioration du module d'extraction de texte. L'objectif est de supporter de nouveaux formats (DOCX, Images via OCR, etc...) et de tester différentes approches pour affiner le calcul de similarité par le traitement des textes du CV, par le traitement du texte de l'offre d'emploi et par l'entraînement de différents algorithmes de DeepLearning.
- **20 Février - 30 Mars :** Fonctionnalités avancées. Implémentation de la comparaison détaillée et optimisation des temps de calcul du LLM.
- **30 Mars - 30 Avril :** Finalisation. Cette dernière phase sera consacrée aux tests utilisateurs, à la correction des bugs résiduels, à la rédaction du rapport final et à la préparation de la soutenance.

7 Prototype réalisé

À l'issue du premier sprint, nous avons mis en place :

Serveur Flask : Routage et requêtes POST fonctionnelles.

Interface Drag & Drop : Sélection multiple de CV intuitive.

Prompting : Champ permettant de détailler les compétences recherchées.

Stockage : Sauvegarde sécurisée des PDF.

Traitement IA : Pipeline complet allant de l'extraction du texte des PDF à la vectorisation.

Visualisation : Des blocs d'information s'affichent pour présenter le contenu extrait et le score de pertinence associé.

Justification des choix techniques :

- **Extraction PDF :** Le choix de **pdfplumber** s'est imposé après nos tests. C'est la librairie qui gère le mieux les mises en page complexes (colonnes, blocs) typiques des CV.
- **Modèle IA :** Nous avons choisi **all-MiniLM-L6-v2**. C'est un modèle très léger qui tourne rapidement en local, ce qui nous permet d'éviter les lenteurs liées aux appels réseau.
- **Interface :** L'association de Flask avec l'API Fetch assure une fluidité maximale pour l'utilisateur.